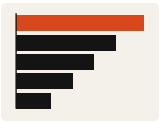
Chart Chooser

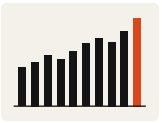
Find what you want to show. Pick the chart. Copy one line.

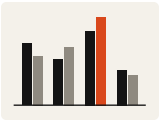
DATAMOUS
Cheat Sheet
Every line tested on matplotlib, seaborn and plotly

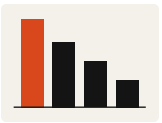
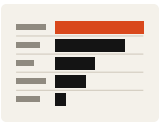
Compare & rank

CHART	USE IT FOR	DATA	CODE

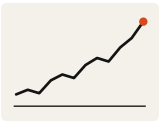
 **Sorted bars**	**Ranking**
	• Products ranked by revenue
	• Sales by store, best to worst
		Category `product`
		Number `revenue`
			matplotlib
			`t = df.sort_values('revenue')`
			`ax.barh(t['product'], t['revenue'])`

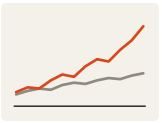
 **Column bars**	**A few items, or a natural order** • Revenue by month • Website visits by weekday	Category `month` Number `revenue`	plotly `px.bar(df, x='month', y='revenue')`

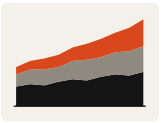
 **Grouped bars**	**Two groupings side by side**
	• Average order by region and segment
	• This year vs last year, by product
		Category `region`
		Category `segment`
		Number `revenue`
			seaborn
			`sns.barplot(data=df, x='region',`
			`y='revenue', hue='segment')`

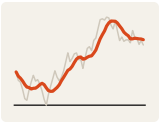
 **Count bars**	**Counting rows** • Number of orders per region • Number of customers per plan type	Category `region`	seaborn `sns.countplot(data=df, x='region')`
 **Table with bars**	**Exact numbers, with bars for size** • Revenue per sales rep • Monthly KPI table in a report	Category `product` Number `revenue`	pandas `df.style.bar(subset=['revenue'])`

Change over time

CHART	USE IT FOR	DATA	CODE
 **Line**	**One trend over time** • Monthly revenue • Daily website visits	Date `month` Number `revenue`	plotly `px.line(df, x='month', y='revenue')`

 **Several lines**	**Comparing trends**
	• Revenue this year vs last year
	• Sales of three products over time
		Date `month`
		Category `year`
		Number `revenue`
			seaborn
			`sns.lineplot(data=df, x='month',`
			`y='revenue', hue='year')`

 **Stacked area**	**A total and its parts over time** • Revenue by sales channel over time • Website traffic by source over time	Date `month` Number `per part`	pandas `df.plot.area(x='month', ax=ax)`

 **Smoothed line**	**The trend in noisy data**
	• Daily sales as a 7-day average
	• Daily sign-ups trend
		Date `date`
		Number `revenue`
			matplotlib
			`s = df.set_index('date')['revenue']`
			`ax.plot(s.rolling(7).mean())`

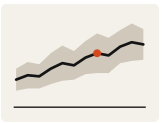
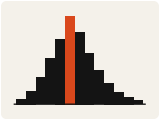
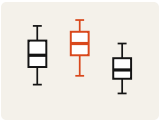
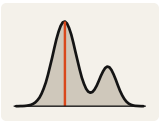
 **Shaded band**	**A range over time**
	• Sales forecast range
	• Daily low and high stock price
		Date `date`
		Number `low`
		Number `high`
			matplotlib
			`ax.fill_between(df['date'],`
			`df['low'], df['high'])`

Chart Chooser

Distribution

CHART	USE IT FOR	DATA	CODE
 **Histogram**	**How values are spread** • Order sizes • Customer ages	Number price	seaborn `sns.histplot(data=df, x='price')`
 **Box plot**	**Spreads by group, outliers** • Prices by region • Delivery times by courier	Category region Number price	plotly `px.box(df, x='region', y='price')`
 **Violin**	**The shape of each group's spread** • Prices by region • Test scores by class	Category region Number price	seaborn `sns.violinplot(data=df, x='region', y='price', cut=0)`
 **Density**	**A smooth outline of one spread** • Delivery times • Time spent on a website	Number price	seaborn `sns.kdeplot(data=df, x='price')`
 **Strip plot**	**Every value, small tables** • Test scores by class • Prices of 20 products by category	Category region Number price	seaborn `sns.stripplot(data=df, x='region', y='price')`

Relationship

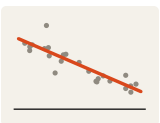
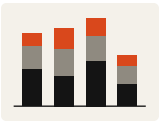
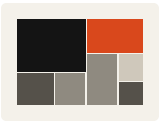
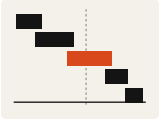
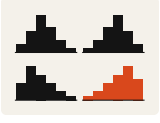
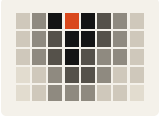
CHART	USE IT FOR	DATA	CODE
 **Scatter**	**Two numbers together** • Price vs units sold • Ad spend vs sales	Number price Number units	plotly `px.scatter(df, x='price', y='units')`
 **Bubble**	**A third number as dot size** • Price vs units sold, sized by revenue • Store size vs sales, sized by staff	Number price Number units Number revenue	seaborn `sns.scatterplot(data=df, x='price', y='units', size='revenue')`
 **Trend line**	**The direction of a relationship** • Price vs units sold, with a fitted line • Ad spend vs revenue	Number price Number units	seaborn `sns.regplot(data=df, x='price', y='units')`
 **Pair plot**	**A first look at many numbers** • All number columns of a new dataset • Price, units and revenue together	Number 3+ columns	seaborn `sns.pairplot(data=df)`
 **Correlation heatmap**	**Which numbers move together** • Correlations between columns • Which columns relate to revenue	Number 3+ columns	seaborn `sns.heatmap(df.corr(numeric_only=True), annot=True)`

Chart Chooser

Part of a whole

CHART	USE IT FOR	DATA	CODE
 **Pie**	**One part's share, 2 to 4 parts** • Online vs in-store sales share • Customers by plan, 2 to 4 plans	Category channel Number revenue	plotly `px.pie(df, names='channel', values='revenue')`
 **Donut**	**A share, with room for a label** • Sales share by channel • Budget split by department	Category channel Number revenue	plotly `px.pie(df, names='channel', values='revenue', hole=0.5)`
 **Stacked bars**	**Totals and their parts** • Sales by region and customer type • Orders by month, split by channel	Category region Category segment Number revenue	plotly `px.bar(df, x='region', y='revenue', color='segment')`
 **Treemap**	**Many parts, in groups** • Revenue by region and product • Budget by department and team	Category region Category product Number revenue	plotly `px.treemap(df, path=['region', 'product'], values='revenue')`
 **Waterfall**	**How a total changes, step by step** • Revenue to profit, cost by cost • Cash from start to end of year	Category step Number amount Category measure	plotly `go.Figure(go.Waterfall(x=df['step'], y=df['amount'], measure=df['measure']))`

Place, process & schedule

CHART	USE IT FOR	DATA	CODE
 **Map**	**Values by place** • Sales by US state • Customers per US state	Category state Number revenue	plotly `px.choropleth(df, locations='state', locationmode='USA-states', scope='usa', color='revenue')`
 **Funnel**	**Where people drop out** • Visits to sign-ups to sales • Job applicants through hiring stages	Category stage Number people	plotly `px.funnel(df, x='people', y='stage')`
 **Timeline**	**Who does what, and when** • Project plan with start and end dates • Marketing campaign schedule	Category task Date start Date finish	plotly `px.timeline(df, x_start='start', x_end='finish', y='task')`
 **Small multiples**	**The same chart for each group** • Price spread for each region • Monthly sales for each store	Category region Number price	plotly `px.histogram(df, x='price', facet_col='region')`
 **Heatmap grid**	**High and low across a grid** • Sales by store and hour • Revenue by month and region	Category store Category hour Number revenue	seaborn `sns.heatmap(df.pivot_table(index='store', columns='hour', values='revenue', aggfunc='sum'))`